Introduction to GitHub Actions

EDC 2026

Alexander Jaust

2026-03-17

Icebreaker

- What is your name?
- Where are you located?
- If you could choose to transform into an animal (and back), what animal would you choose?

Goals & Outcomes

- Read and write GitHub workflows
- Have understanding workflow files
- Use pre-defined Actions
- Use matrix builds and multi-OS testing
- Have some fundamental knowledge about security
- Create downloadable artifacts and use Docker containers

Schedule

- 10:00: Start of workshop
- Breaks as we need them
- 13:30: End of workshop

Logistics & Prerequisites

- GitHub account, basic git (clone/commit/push)
- Laptop with browser, VS Code (or editor)
- Codespaces recommended
- Starter repo with sample code and tests

What is “GitHub Actions”?

> *GitHub Actions is a continuous integration and continuous delivery (CI/CD) platform that allows you to automate your build, test, and deployment pipeline. You can create workflows that build and test every pull request to your repository, or deploy merged pull requests to production.*

From: docs.github.com/en/actions/get-started/understand-github-actions

Exercise 0: Initial Setup

- Create a copy or fork of the workshop repository

https://github.com/ajaust/introduction-to-github-actions-edc26

If you create a fork, copy all branches. There is a `solution` branch.

Exercise 1: Create Your First Workflow

- Create the `first_workflow.yml` file in repo
- Commit and push
- Open Actions tab and inspect the run

Anatomy of a workflow file

```yaml
name: My first GitHub workflow
run-name: First workflow # Optional, modifies appearance on GitHub

# Events triggering the workflow
on: [push]

# Group of jobs to run as part of the workflow
jobs:
  # A job
  greet:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Hello world"
```

Workflow triggers

- Common events (some with activity types)
  - `push`
  - `pull_request`
  - `schedule`: Scheduled using `cron` syntax
  - `workflow_dispatch`: Trigger workflow manually from your repository
- Can combined with branches (2 ways to specify list of branches)

Docs: Events that trigger workflows

```yaml
on:
  push:
    branches: [main]
  pull_request:
    branches:
      - main
  workflow_dispatch:
```

Executing jobs (1/2)

- A workflow can have several jobs
- Jobs have a `job_id` (`job_one`, `job_two`)
- Jobs run on a runner, we will stick to GitHub-hosted runners
  - `ubuntu-latest` (Linux)
  - `macos-latest`
  - `windows-latest`
- Jobs can have names

Docs: Using jobs in a workflow

Docs: GitHub-hosted runners

```yaml
[...]
jobs:
  job_one:
    runs-on: ubuntu-latest
    steps:
      - name: "Steps can have a name"
        run: echo "Hello world"
  job_two:
    runs-on: macos-latest
    name: "Job with a name"
    steps:
      - name: "Step with multi-line commands"
        run: |
        echo "Hello from"
        echo "job_two"
```

Executing jobs (2/2)

- A job can contain many steps
- Jobs can depend on each other via `needs: <job_id>`
- Jobs can have timeouts

Docs: Using jobs in a workflow

Docs: GitHub-hosted runners

```yaml
[...]
jobs:
  multistep_job:
    timeout-minutes: 5
    runs-on: ubuntu-latest
    steps:
      - name: "Steps can have a name"
        run: echo "Hello world"
      - name: "So sleepy... zzzZZZzz"
        run: sleep 30 # sleeps for 30 seconds
  dependent_job:
    needs: multistep_job
    runs-on: windows-latest
    steps:
      - run: echo "Job on windows runner"
```

Exercise 2: Multi-job workflows and failing jobs

Variables, expressions and contexts

- Evaluate an expression using `${{ EXPRESSION }}`
- Many contexts exists `github`, `env`, `job`, `secrets`...
- Can be combined with conditionals `if: CONDITION`
  - Can be used on different levels: `job`-level, `step`-level

Docs: Variables
Docs: Expressions
Docs: Contexts

```yaml
env:
  MyVariable: "Hello world"
[...]
jobs:
  [...]
  job_id:
    env:
      JobVariable: "This is a job variable"
    steps:
    - run: echo "${{ env.MyVariable }}"
    - run: echo "${MyVariable}"
    - name: Evaluate "github.action"
      run: echo "${{ github.action }}"
    - name: "Print matrix version"
      if: ${{ matrix.python-version == '3.14' }}
      run: echo "${{ matrix.python-version }}"
```

Debugging GitHub workflows

- Quite simple to add mistakes, quite hard to catch early
- Hard to run actions locally, e.g, via `act`
- Good solution: actionlint is a "Static checker for GitHub Actions workflow files"
  - Is preinstalled on our Codespace
  - `https://github.com/nektos/act`
- Hard to run actions locally, e.g, via `act`

Exercise 3: Variables and secrets

Strategies (matrix jobs)

- On can use premade Actions
- Actions in `actions/` are maintained by GitHub
- `actions/checkout` checks out the repository
- `actions/setup-python` configures a Python version
  - Similar actions exist for other environments, e.g., Node/Javascript etc.

Docs: Using pre-written building blocks in your workflow

GitHub Action marketplace

```yaml
[...]
jobs:
  [...]
  run-code:
    runs-on: ${{ matrix.os }}
    strategy:
      fail-fast: false
      matrix:
        python-version: [ "3.12", "3.13" ]
        os: [ "macos-latest", "ubuntu-latest"
```

Building on top of other Actions

- On can use premade Actions
- Actions in `actions/` are maintained by GitHub
- `actions/checkout` checks out the repository
- `actions/setup-python` configures a Python version
  - Similar actions exist for other environments, e.g., Node/Javascript etc.

Docs: Using pre-written building blocks in your workflow GitHub Action marketplace

```yaml
[...]
job:
  run-code:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6

      - uses: actions/setup-python@v6
        with:
          python-version: '3.14'

      - run: python mycode.py
```

Exercise 4: Matrix builds

Security: Permissions

- Workflow contains `GITHUB_TOKEN` (`secrets.GITHUB_TOKEN`)
  - Can be used to use GitHub API from within workflow
- Explicitly set permissions, default permissions often too broad

```yaml
run-name: First workflow # Optional
permissions: {}

[...]
run-code:
  runs-on: ubuntu-latest
  permissions:
    contents: read
```

Docs: Secure use

Th & Fr: "Pimp your GHAS" sessions by Nicole

Security: Third-party actions

- Pin the action to a commit hash, e.g., commit hash of a release

```yaml
steps:
  - uses: actions/checkout@v5
  - uses: hadolint/hadolint-action@2332a7b74a6de0dda2e2221d575162eba76ba5e5
    with:
      dockerfile: Dockerfile
```

Equinor AppSec on GitHub Actions

Hadolint a linter for Dockerfiles

Exercise 5: Security

Costs

- Let's inspect the costs panel in our GitHub account
  - Your accounts -> Settings -> Billing and licensing
- GitHub covers costs of public repositories
- Equinor covers costs of Equinor repositoris
  - **Not visible in your account**

Fr. 10:00: "How we got to be the biggest spenders of GitHub CI minutes within the company" by Andreas

Docker containers

```yaml
jobs:
  # Label of the container job
  container-job:
    # Containers must run in Linux based operating systems
    runs-on: ubuntu-latest
    # Docker Hub image that `container-job` executes in
    container: python:3.14-slim
```

Docs: Using containerized services

Artifacts

```yaml
- name: "Create new file"
  run: echo "Hello world!" >> new_file.txt

- name: "Upload Artifact"
  uses: actions/upload-artifact@v7
  with:
    name: my-artifact
    path: new_file.txt
    retention-days: 5
```

Docs: Workflow artifacts

Exercise 6: Creating artifacts and using Docker

Further topics

- Slack integration
- GitHub Action market place: https://github.com/marketplace and choose `Actions -> All Actions` on the left.
  - Be careful what Actions to choose. They may have access to your repository, secrets etc.
- Reusable workflows
- Publication of software packages, e.g., Python wheels to PyPI
- Using self-hosted runners

Q&A and Feedback

- Feedback form will be posted on Slack